

Supplementary Material A: Routing Improvements for the Hybrid LLM–Neural-Network System Impact on DeFi Extrapolation Benchmark Performance

Technical Note — Hybrid System Development Log

February–March 2026

Note. Figures in this document that could not be reproduced from released result files have been removed. The removed material and its provenance are recorded in the Post-Submission Report.

Contents

1	Background and Motivation	2
2	Data-Generation Fixes	3
2.1	Reserve Ratio and Spot Price (Fix 0)	3
2.2	Impermanent Loss Breakeven (Fix 0b)	3
3	Routing Improvements (Fixes 1–4)	4
3.1	Fix 1: Extrapolation Probe Degradation	4
3.2	Fix 2: Formula Complexity Routing	5
3.3	Fix 3: LLM Predictions as a Neural-Network Feature	5
3.4	Fix 4: Distance-Gated Blend Weights	6
4	Fix 5: Unified Formula Evaluator and Routing Guard	6
4.1	Root Cause: Divergent Evaluation Paths	6
4.2	Fix 5: Unified Evaluator	7
4.3	Routing Override Guard (Fix 5b)	7
5	Impact on the Leaderboard	8
6	Interaction Between Fixes	8
7	Summary of Changes	8
A	Complete Proof of the Main Paper’s <i>LLM Reproductive Behaviour</i> Finding	9
B	Reproducibility Statement	10
B.1	Code Repository	10
B.2	Running Experiments	10

B.3	Hardware and Software	11
B.4	Random Seeds	12
B.5	Measurement Correction (v1 $\rightarrow$ v2)	12
B.6	Troubleshooting	12
B.7	Reproducibility Checklist	13
B.8	Contact	13
C	Computational Complexity Analysis	14
C.1	Theoretical Complexity	14
C.2	Measured Runtime	14
D	Extended Discussion	14
D.1	Why LLMs Fail on Novel Domains	14
D.2	Future Research Directions	16
D.3	Final Thoughts	18

Note to reader. This document is Supplementary Material A to the main JMLR submission *HypatiaX: A Hybrid Symbolic-Neural Framework for Extrapolation-Reliable Analytical Discovery* (`jmlr_paper_main.tex`). It provides full implementation details for the six routing fixes summarised in the main paper’s *Component 3: Five-Stage Routing and Ensembling* subsection—which now includes Proposition 1 (Routing Cascade Convergence), a formal guarantee that Phase 2 warm-starting is monotonically non-degrading over the Pareto front—and reports cumulative impact on the DeFi extrapolation benchmark.

1 Background and Motivation

Remark 1 (Development-set status) *The DeFi equations used to motivate and validate the routing fixes in this document (the main paper’s Benchmark Design section) overlap with equations in the Core 15 benchmark and the DeFi 74-case extrapolation suite (v3.0). Post-fix performance figures on the DeFi suite should therefore be interpreted as in-sample routing improvement, not held-out generalisation. This caveat is disclosed here; no equivalently-numbered remark exists in the current main paper draft, so it should not be cited as appearing there.*

The DeFi extrapolation benchmark evaluates three methods — *Pure LLM*, *Neural Network (NN)*, and *Hybrid* — across 74 test cases ranging in difficulty from easy linear formulas to hard transcendental functions. Each case splits data so the test set lies *outside* the training feature range, directly probing extrapolation ability rather than interpolation.

Prior to the improvements described here, the aggregate picture was:

Post-mortem analysis identified a root cause: **routing confidence**. The hybrid’s decision to assign a case to the NN or blended ensemble relied on in-distribution training R^2 as the sole criterion, which is a poor proxy for extrapolation quality.

Two additional data-generation bugs contributed:

- **Reserve Ratio / Spot Price (zero-variance targets).** Both cases generated `reserve_x` and `reserve_y` from identical `linspace` ranges, producing a ratio ≈ 1.0 with negligible variance.

Table 1: Baseline performance before routing improvements (standard cases, intractable cases excluded; denominator $n=66$, pre-v3.0 historical). The Hybrid $R^2 > 0.99$ figure is not reported: it could not be traced to a source file. The v3.0 benchmark uses $n = 74$; its own $R^2 > 0.99$ figure is **59.5 %** (44/74, decision-attribution-bug-corrected — see main paper, § “The Hybrid Decision-Attribution Bug”).

Method	Median R^2	$R^2 > 0.9$ (%)	$R^2 > 0.99$ (%)	Catastrophic
Pure LLM	1.000	85.7	83.9	4
Neural Net	0.752	27.9	1.5	6
Hybrid	1.000	—	<i>removed</i>	1

- **Impermanent Loss Breakeven (misclassified as tractable).** This case produced catastrophic NN scores ($R^2 \approx -41\,000$) and a degenerate LLM artefact ($R^2 = 1.000$, a train/test split artefact).

2 Data-Generation Fixes

2.1 Reserve Ratio and Spot Price (Fix 0)

```

1 reserve_x = np.linspace(100, 100000, n)
2 reserve_y = np.linspace(100, 100000, n)    # same range!
3 reserve_ratio = reserve_x / (reserve_y + 1e-10)  # ~1.0 everywhere
    
```

Listing 1: Buggy generator — identical linspace ranges.

```

1 rng = np.random.default_rng(7)
2 reserve_x = np.exp(rng.uniform(np.log(100), np.log(100000), n))
3 reserve_y = np.exp(rng.uniform(np.log(100), np.log(100000), n))
4 reserve_ratio = reserve_x / reserve_y
5 # std(reserve_ratio) ~ 26.4, range: [0.002, 212.6]
    
```

Listing 2: Fixed generator — independent log-uniform sampling (seed=7).

The same fix applies to the Spot Price generator (`reserve_a`, `reserve_b`). After the fix, both cases present a genuine regression target.

2.2 Impermanent Loss Breakeven (Fix 0b)

```

1 {
2     "name": "Impermanent loss breakeven",
3     "domain": "liquidity",
4     "extrapolation_intractable": True,
5     "intractable_reason":
6         "Breakeven condition involves implicit solving; aggressive
          high-split "
7     "puts test set in a degenerate regime. NN R2 collapses
          catastrophically; "
8     "LLM 1.000 is a split artefact."
    
```

9 }

Listing 3: Intractable flag added to IL Breakeven catalogue entry.

Excluding this case from the standard aggregate removes the last remaining catastrophic case from the Hybrid column and avoids inflating the LLM advantage.

3 Routing Improvements (Fixes 1–4)

All four routing improvements are implemented as methods of the `EnhancedExtrapolationTest` class and fire *after* the hybrid’s own training-time decision, potentially overriding it before test-set evaluation. The override priority order is:

1. Fix 2 (formula complexity) — cheapest, no extra training.
2. Fix 1 (extrapolation probe) — if Fix 2 did not already override.
3. Fix 3 (LLM feature augmentation) — when decision remains "nn".
4. Fix 4 (distance-gated blend weight) — in "ensemble" paths.

3.1 Fix 1: Extrapolation Probe Degradation

Motivation. A neural network fitting training data well may still extrapolate poorly. In-distribution R^2 cannot detect this.

Method.

1. Sort training samples by the primary feature.
2. Reserve the top $p = 15\%$ as a *probe set*.
3. Train a small MLP (layers: [64, 32], 200 epochs, Adam) on the remaining $1-p$ fraction.
4. Compute $\Delta R^2 = R_{\text{in}}^2 - R_{\text{probe}}^2$. If $\Delta R^2 \geq 0.15$, override to "llm".

```

1 def _extrapolation_probe_degradation(self, X_train, y_train,
2   p=0.15):
3     n = len(X_train)
4     split = int(n * (1 - p))
5     order = np.argsort(X_train[:, 0])
6     X_sorted, y_sorted = X_train[order], y_train[order]
7
8     X_fit, y_fit = X_sorted[:split], y_sorted[:split]
9     X_probe, y_probe = X_sorted[split:], y_sorted[split:]
10
11     # ... train MLP on (X_fit, y_fit) ...
12
13     degradation = max(0.0, r2_in - r2_probe)
14     return degradation # caller: if degradation >= 0.15 ->
15         route to LLM

```

Listing 4: Extrapolation probe core logic.

Measured gain. +6 percentage points on $R^2 > 0.99$.

3.2 Fix 2: Formula Complexity Routing

Motivation. The LLM’s extracted formula is free information about the functional class of the target. Transcendental or piecewise operations make NN extrapolation structurally unreliable regardless of training R^2 .

Method.

```

1 _TRANSCENDENTAL_TOKENS = [
2     "math.exp", "np.exp", "exp(",
3     "math.log", "np.log", "log(",
4     "math.sqrt", "np.sqrt", "sqrt(",
5     "norm.cdf", "norm.pdf", "scipy.stats",
6     "np.maximum", "np.minimum", "max(", "min(",
7     "math.sin", "np.sin", "math.cos", "np.cos",
8     "**0.",      # fractional powers
9 ]
    
```

Listing 5: Transcendental token list.

Any match forces an immediate override to "llm" without running the probe (cheaper, deterministic). Fix 2 is checked before Fix 1.

Measured gain. +5 percentage points.

3.3 Fix 3: LLM Predictions as a Neural-Network Feature

Motivation. When routing remains "nn", the NN must infer the functional form from scratch. Concatenating LLM predictions as an extra input feature forces the NN to learn residuals rather than the full function.

Method.

$$\mathbf{X}_{\text{aug}} = [\mathbf{X} \mid \hat{y}_{\text{LLM}}] \in \mathbb{R}^{n \times (d+1)}.$$

```

1 llm_pred_train = execute_python_code_get_predictions(llm_code,
2     X_train)
3 llm_pred_test  = execute_python_code_get_predictions(llm_code,
4     X_test)
5 X_train_aug = np.column_stack([X_train, llm_pred_train])
6 X_test_aug  = np.column_stack([X_test,  llm_pred_test])
7 nn_m = self._train_and_eval_nn(X_train_aug, y_train, X_test_aug,
8     y_test)
    
```

Listing 6: LLM-feature augmentation.

Guard condition: requires positive LLM training R^2 ; otherwise plain NN.

Measured gain. +1 percentage point.

3.4 Fix 4: Distance-Gated Blend Weights

Motivation. In the "ensemble" path, the LLM-NN blend weight was fixed at training time. In an extrapolation benchmark the test set is always outside the training range by construction.

Method. Define the out-of-range fraction:

$$\rho = \frac{1}{m} \sum_{i=1}^m \mathbf{1} \left[\exists j : x_{ij} < \min_k x_{kj}^{\text{train}} \text{ or } x_{ij} > \max_k x_{kj}^{\text{train}} \right].$$

The LLM blend weight becomes:

$$w_{\text{LLM}} = w_0 + (1 - w_0) \rho, \quad w_0 = 0.3,$$

so $w_{\text{LLM}} \in [0.3, 1.0]$. Since all extrapolation test sets satisfy $\rho \approx 1$, the blend collapses toward pure LLM.

```

1 def _distance_llm_weight(X_test, X_train, base_weight=0.3):
2     train_min = X_train.min(axis=0)
3     train_max = X_train.max(axis=0)
4     outside   = np.mean(
5         np.any((X_test < train_min) | (X_test > train_max),
6             axis=1)
7     )
8     return float(base_weight + (1.0 - base_weight) * outside)

```

Listing 7: Distance-gated blend weight.

Measured gain. +1 percentage point.

4 Fix 5: Unified Formula Evaluator and Routing Guard

4.1 Root Cause: Divergent Evaluation Paths

Two cases returned catastrophic Hybrid scores even though routing was correct and the LLM formula was valid:

- **Correlated Portfolio VaR:** Pure LLM = +1.000, Hybrid = −12.121.
- **Impermanent loss in constant product:** Pure LLM = +1.000, Hybrid = NaN.

The Hybrid’s test-set path called `hybrid.evaluate_llm_formula()`, while the Pure LLM path called `PureLLMBaseline.test_formula_accuracy()`. On formulas involving matrix operations or multi-variable lookups, the two namespace implementations diverged silently.

4.2 Fix 5: Unified Evaluator

```

1 @staticmethod
2 def _eval_formula_r2(llm_code, X, y_true, var_names):
3     try:
4         from
5             hypatiax.experiments.tests.hybrid_ensemble_system_defi_domain
6             import (
7                 execute_python_code_get_predictions)
8         y_pred = execute_python_code_get_predictions(llm_code, X)
9         if y_pred is None or np.any(np.isnan(y_pred)) or
10            np.any(np.isinf(y_pred)):
11             return float("nan"), False
12         ss_res = np.sum((y_true - y_pred) ** 2)
13         ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
14         r2 = float(1 - ss_res / ss_tot) if ss_tot > 1e-10 else 0.0
15         return r2, True
16     except Exception:
17         return float("nan"), False

```

Listing 8: Unified formula evaluator (Fix 5).

4.3 Routing Override Guard (Fix 5b)

Two further cases exposed a related failure: Fixes 1 and 2 overrode to "llm" even when the LLM formula had negative training R^2 .

```

1 _llm_train_r2 = float(
2     train_result.get("llm_result", {})
3     .get("metrics", {}).get("r2", 0.0) or 0.0
4 )
5 _llm_formula_trustworthy = has_formula and (_llm_train_r2 > 0.0)
6
7 # Fixes 1 and 2 only fire when the formula actually fit training
8 # data
9 if _llm_formula_trustworthy and decision in ("nn", "ensemble"):
10     if self._formula_has_transcendental(llm_code): # Fix 2
11         decision = "llm"
12
13 if _llm_formula_trustworthy and decision in ("nn", "ensemble"):
14     if degradation >= 0.15: # Fix 1
15         decision = "llm"

```

Listing 9: LLM-formula trustworthiness guard.

The same guard applies to Fix 3: if the LLM formula has negative training R^2 , augmentation is skipped.

Table 2: Routing fixes in order of application. No per-fix percentage impact is reported: the figures previously shown here could not be traced to a source file. The decision-attribution-bug-corrected near-perfect rate on the DeFi benchmark is **59.5 %** (44/74), reproduced by re-running the correction rule against the canonical seed-42 file (md5 cbf503d0b3985b4c1bccf7ed18e0316b); the uncorrected rate is **90.5 %** (67/74). See the main paper (jmlr_paper_main.tex). Each fix below is a verified code change (§ 7).

Fix	Description
Baseline	In-dist. R^2 routing (old)
Fix 0	Reserve Ratio / Spot Price: independent log-uniform sampling
Fix 0b	IL Breakeven flagged intractable
Fix 1	Extrapolation probe routing
Fix 2	Formula complexity routing
Fix 3	LLM feature augmentation
Fix 4	Distance-gated blend weight
Fix 5 (proj.)	Unified evaluator + routing guard

5 Impact on the Leaderboard

Each fix above addresses a distinct, verified routing or data-generation defect (§ 7); their individual and cumulative effect on the near-perfect rate is not reported. See the main paper for the current, corrected comparison (59.5%) and its caveats.

6 Interaction Between Fixes

- **Fix 2 subsumes a subset of Fix 1.** Cases with transcendental tokens are caught by Fix 2 before Fix 1 runs, avoiding the probe NN cost.
- **Fix 3 benefits from Fix 1’s precision.** Cases remaining "nn" after the probe are safer for LLM-feature augmentation.
- **Fix 4 becomes redundant for cases upgraded by Fixes 1/2.** Fix 4’s value lies in residual ensemble cases that neither Fix 1 nor Fix 2 catches.
- **Data Fix 0 is orthogonal to routing.** Its contribution is purely additive.

7 Summary of Changes

The main paper’s decision-attribution-corrected near-perfect rate is 59.5% (44/74), tied with Pure LLM’s own corrected rate on the same benchmark (see main paper, § “The Hybrid Decision-Attribution Bug”). The routing fixes documented in this supplement (§ 7) are each independently verified code changes; their individual or cumulative effect on the near-perfect success rate is not reported.

Table 3: All changes applied across the two modified files. Descriptive inventory of code changes only; it contains no performance figures or estimates.

File	Fix	Location	Description
experiment_protocol_defi.py	0	Test 5	Reserve Ratio: ind
	0	Test 7	Spot Price: indepe
test_enhanced_defi_extrapolation.py	0b	Catalogue	IL Breakeven flagg
	–	_load_checkpoint	Deduplication by t
	1	_extrapolation_probe_degradation	Probe method add
	2	_formula_has_transcendental	Token list + check
	1,2	test_method (hybrid block)	Override logic inje
	3	test_method (nn branch)	LLM-feature augm
	4	test_method (ensemble branch)	Distance-gated ble
	–	_distance_llm_weight	Blend weight help

Appendix A. Complete Proof of the Main Paper’s *LLM Reproductive Behaviour* Finding

Proof [Detailed Proof] Let M_θ be an autoregressive language model with parameters θ trained on corpus $\mathcal{C}$. Expression $E = t_1 \cdots t_n$ is generated via:

$$P_\theta(E \mid D) = \prod_{i=1}^n P_\theta(t_i \mid t_{<i}, D)$$

where $t_{<i} = t_1, \dots, t_{i-1}$ is the prefix and D represents conditioning data.

Step 1: Training concentrates probability mass. Maximum likelihood training optimizes:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{E \sim \mathcal{C}} [\log P_\theta(E)]$$

This causes $P_{\theta^*}(t_i \mid t_{<i}, D)$ to concentrate on tokens frequently observed in $\mathcal{C}$ given context $t_{<i}$.

Step 2: Novel tokens incur exponential penalty. For expression $E \notin \mathcal{C}$, define:

$$d(E, \mathcal{C}) = \min_{E' \in \mathcal{C}} d_{\text{edit}}(E, E')$$

Let $\mathcal{N}(E) \subseteq \{1, \dots, n\}$ denote positions where t_i is novel (low probability given $t_{<i}$). Under independence approximation:

$$P_\theta(E \mid D) \approx \prod_{i \in \mathcal{N}(E)} p_i \cdot \prod_{i \notin \mathcal{N}(E)} p'_i$$

where $p_i \ll 1$ for novel tokens and $p'_i \approx 1$ for familiar tokens.

Step 3: Exponential decay. Since $|\mathcal{N}(E)| \geq d(E, \mathcal{C})$ and $p_i \leq p_{\max} < 1$:

$$P_{\theta}(E \mid D) \leq p_{\max}^{d(E, \mathcal{C})} = \exp(-\beta \cdot d(E, \mathcal{C}))$$

where $\beta = -\log p_{\max} > 0$. Thus generation probability decays exponentially with edit distance from training data, establishing reproductive behavior. ■ ■

Remark on rigour: This proof assumes approximate local independence of token probabilities. A fully rigorous treatment would analyse the joint distribution over all tokens. The exponential decay result is a consequence of the independence approximation; empirical results are consistent with this theoretical prediction, but the approximation means the proof should be understood as providing intuition rather than a formal guarantee (Kambhampati et al., 2024).

Appendix B. Reproducibility Statement

All experimental data, source code, and configuration files are publicly available to ensure full reproducibility of our results.

B.1 Code Repository

GitHub (reproduction; all tests and benchmarks): <https://github.com/sednabcn/LLM-HypatiaX-REPRO>

GitHub (post-processing and pre-submission): <https://github.com/sednabcn/LLM-HypatiaX-DEV>

Repository Structure (reproduction repository):

```
LLM-HypatiaX-REPRO/
+-- hypatiax/
|   +-- analysis/           # Statistical analysis
|   +-- core/
|       +-- base_pure_llm/  # Pure LLM baseline
|       +-- generation/    # Hybrid systems
|       +-- training/      # Neural network baseline
|   +-- data/results/      # Raw experimental data
|   +-- experiments/
|       +-- benchmarks/    # Feynman, noise-sweep, sample-complexity runners
|       +-- tests/         # DeFi extrapolation tests
|   +-- protocols/        # Experimental protocols
|   +-- tools/
+-- .github/               # CI workflows and analysis scripts
+-- config/repro.yaml
+-- docs/
+-- AUDIT/
+-- run_all.sh
+-- requirements.txt
```

B.2 Running Experiments

Quick Start (all v2-corrected commands):

```
git clone https://github.com/sednabcn/LLM-HypatiaX-REPRO
cd LLM-HypatiaX-REPRO
pip install -r requirements.txt
source activate_hypatiaux.sh

# Feynman SR --- Phase 3 (§5.8, ~1h 13min on reference hardware)
python hypatiaux/experiments/benchmarks/\
run_comparative_suite_benchmark_v2.py \
    --noiseless --threshold 0.999999 \
    --nn-seeds 3 --samples 200 \
    --method-timeout 900 --pysr-timeout 1100
```

Other runners in the repository: `hypatiaux/experiments/benchmarks/run_noise_sweep_benchmark.py`, `run_sample_complexity_benchmark.py`, `hypatiaux/experiments/tests/test_enhanced_defi_extrapolation.py` (DeFi extrapolation), and `run_all.sh` at the repository root. The Phase 3 script identifies itself as `run_protocol_benchmark_core.py` in its docstring; its output files are named `protocol_core_*`.

B.3 Hardware and Software

All experiments were run on a single machine: Intel Celeron T3100 @ 1.90 GHz (2 cores), 3 GB DDR2 RAM, no GPU, Kali GNU/Linux Rolling. Runtimes on modern multi-core hardware will be substantially lower; a GPU is *not* required to reproduce the core results.

Table 4: Software environment (verified 18 March 2026).

Component	Version	Note
Python	3.12.2	
PySR	1.5.9	
Julia	1.12.2	required by PySR
juliacall	0.9.26	Python↔Julia bridge
NumPy	2.3.5	
SciPy	1.16.3	
SymPy	1.14.0	
Pandas	2.3.3	
Matplotlib	3.10.7	
Seaborn	0.13.2	
scikit-learn	1.7.2	
Anthropic SDK	0.73.0	claude-sonnet-5, temp 0.25
PySR populations	2	hardware limit (2-core CPU)

Expected Wall-Clock Times (reference hardware):

Experiment	Tests	Wall Time
Pure LLM Baseline	40	5–10 min
Pure Symbolic (Core 15)	18	117 min
LLM-Guided Hybrid	30	35 min
DeFi Suite (73 cases)	23	44 min
Hybrid LLM+NN (v50_2)	30	45 min
Feynman SR Phase 3	30	~73 min
Statistical Analysis	—	15 min
Figure Generation (13)	—	10 min
All Experiments	131	~6 hours

B.4 Random Seeds

- All campaigns: base seed 42 (`BenchmarkProtocol.seed = 42`)
- Per-equation offset: `seed + index × 7`
- LLM calls: temperature 0.25 for all methods
- Neural network (Phase 3): 3 independent seeds per equation; median R^2 reported

B.5 Measurement Correction (v1 → v2)

A bug in `evaluate_llm_formula` was corrected on 2026-03-16. The function used a hard-coded absolute threshold (`ss_tot > 1e-10`) that silently returned $R^2 = 0$ for equations with outputs far below unity. The fix scales the threshold relative to the data magnitude. All results in this paper use the v2 corrected harness. See the *Feynman Benchmark Reproducibility Statement* appendix and the *Measurement Correction (v1→v2)* section of Supplementary B (*HypatiaX: Noise Robustness, Sample Complexity, and Convergence of Hybrid LLM-Neural Symbolic Regression on the Feynman Benchmark*) for the exact code change and affected equations.

B.6 Troubleshooting

PySR / Julia installation:

```
# Install Julia 1.12.2 (required)
wget https://julialang.org/downloads/ # follow official installer
export PATH="$HOME/.juliaup/bin:$PATH"

# Install PySR
pip install pysr==1.5.9
python -c "import pysr; pysr.install()"
```

Arrhenius hang (Feynman test 4): Always use `--method-timeout 900`. Without it, PySR enters an infinite search on this single-variable flat-landscape equation. This is a known Julia JIT behaviour on 2-core hardware; counted as a genuine failure in the paper results.

LLM API:

```
export ANTHROPIC_API_KEY="your-api-key-here"
```

Memory on constrained hardware:

```
# In hypatiaux/tools/symbolic/hybrid_system_v50_2.py
# populations=2 (hardware-limited default used in this paper)
```

B.7 Reproducibility Checklist

1. Clone repository and install dependencies (including Julia 1.12.2)
2. Set ANTHROPIC_API_KEY for LLM-dependent runners
3. Run Core 15 benchmark — confirm the Core 15 far- R^2 Mann-Whitney values in the main paper ($U_H = 72$, $p = 0.095$ with failed cells ranked worst; $p = 0.60$ and 0.43 under the two exclusion conventions; recomputed from the raw ablation shards `exp1_ablation_results_shard0--3.json`, paired Wilcoxon signed-rank on the 10 equations with finite far- R^2 in both arms: $W = 9$, $p = 0.25$; none significant at the 5% level)
4. Run DeFi benchmark — confirm HypatiaX’s decision-attribution-bug-corrected near-perfect rate of **59.5%** (44/74, re-verified 2026-09-27 against the canonical seed-42 file); see the main paper’s Hybrid Decision-Attribution Bug section
5. Run Feynman Phase 3 — confirm Hybrid DeFi 96.7% ($R^2 > 0.9999$)
6. Run statistical analysis — confirm Cohen’s $d = 0.95$ (pooled)
7. Regenerate 13 figures — confirm visual match with paper
8. Verify v2 result files match checksums in repository README

B.8 Contact

- **GitHub Issues:** <https://github.com/sednabcn/LLM-HypatiaX-REPRO/issues>
- **Email:** ruperto.bonet@modelphysmat.com
- **Response time:** Within 2 weeks
- **Documentation:** `docs/QUICK_START_GUIDE.md`

We maintain the repository and welcome contributions, bug reports, and extension requests.

Appendix C. Computational Complexity Analysis

C.1 Theoretical Complexity

Complexity Result 1 (HypatiaX Complexity) *Let n be candidate expressions, m evaluation points, k symbolic constraints. Total complexity is $\mathcal{O}(n \cdot (k + m))$.*

Proof For each candidate expression E :

1. **Symbolic constraint checking:** $\mathcal{O}(k)$ operations (dimensional analysis, domain rules)
2. **Numerical evaluation:** $\mathcal{O}(m)$ operations (fitness computation over dataset)

Total per candidate: $\mathcal{O}(k + m)$. With n candidates: $\mathcal{O}(n(k + m))$. ■ ■

Comparison with Neural Networks:

- **Neural Network Training:** $\mathcal{O}(e \cdot b \cdot (d \cdot h + h^2))$ where e = epochs, b = batch size, d = input dimension, h = hidden units
- **Typical values:** $e = 200, b = 160, d = 3, h = 64 \Rightarrow \mathcal{O}(1.4 \times 10^8)$ operations
- **HypatiaX:** $n = 1000, m = 160, k = 50 \Rightarrow \mathcal{O}(2 \times 10^5)$ operations per iteration

C.2 Measured Runtime

Runtimes are reported only from the mechanical regeneration of the runtime table from the five 74-task per-seed v3 result files in the repository (main paper, Section “Runtime Analysis”, Table `tab:timing_full`). The hybrid system is $3.8\text{--}11.2\times$ *slower* than the neural baseline on every seed, never faster; mean per-task neural-network time is 0.27 s pooled over 370 task runs (per-seed means 0.18–0.36 s). LLM-guided mode routes 68 of 74 cases (92 %) directly to the LLM formula, bypassing full PySR search (68 of 74 for the seed-42 file; 68–69 in each of the five seed files). No stage-level timing breakdown, dataset-size scalability sweep or runtime-versus-accuracy table is reported here, because none could be regenerated from a result file; see the Post-Submission Report.

Appendix D. Extended Discussion

D.1 Why LLMs Fail on Novel Domains

Our empirical results (see the Pure LLM baseline discussion in the main paper) raise a question: **Why does LLM performance decline on specialised domains?** (Kambhampati et al., 2024)

Training Data Coverage Hypothesis: Classical physics formulas appear extensively in:

- **Textbooks:** Digitized and crawled by Common Crawl, Books3 corpus

- **Wikipedia:** High-quality articles with LaTeX equation markup
- **Academic papers:** Often reproduce fundamental equations in introduction sections
- **Code repositories:** Physics simulations, homework solutions (GitHub, GitLab)
- **Q&A forums:** StackExchange Physics, Mathematics, Computational Science
- **Educational websites:** Khan Academy, HyperPhysics, Physics Classroom

Estimated training exposure: 10^6 - 10^7 occurrences for common equations like $F = ma$, $E = mc^2$, $PV = nRT$.

DeFi formulas, by contrast:

- **Emerged post-2018:** After many LLM training data cutoffs (GPT-3: Sept 2021)(Kambhampati et al., 2024)
- **Limited sources:** Primarily white papers, technical docs, GitHub repos
- **Non-standard notation:** $x \cdot y = k$ vs traditional mathematical conventions
- **Niche audience:** Cryptocurrency/blockchain community, not mainstream education
- **Rapid evolution:** Formulas change as protocols update (Uniswap v2 $\rightarrow$ v3 $\rightarrow$ v4)

Estimated training exposure: 10^2 - 10^3 occurrences for common DeFi equations.

Coverage Hypothesis (illustrative estimates): The following order-of-magnitude figures are rough estimates, not measured counts:

Table 5: Equation Prevalence in Training Data (estimated)

Equation	Domain	Approx. Occurrences
$F = ma$	Classical Physics	$\sim 5 \times 10^6$
$E = \frac{1}{2}mv^2$	Classical Physics	$\sim 2 \times 10^6$
$PV = nRT$	Classical Physics	$\sim 1 \times 10^6$
$k = A \exp(-E_a/RT)$	Chemistry	$\sim 5 \times 10^4$
$\text{pH} = \text{pKa} + \log([A^-]/[HA])$	Chemistry	$\sim 2 \times 10^4$
$x \cdot y = k$	DeFi	$\sim 3 \times 10^2$
$\text{IL} = 2\sqrt{r}/(1+r) - 1$	DeFi	~ 50
$\text{VaR} = P\sigma z$	Finance (general)	$\sim 1 \times 10^4$

Conceptual Complexity Comparison: Beyond data availability, specialized domains require distinct reasoning patterns:

Table 6: Conceptual Requirements Comparison. [Qualitative author assessment; not derived from measured data.]

Requirement	Classical Physics	DeFi/Risk
Distributional reasoning	Rare	Essential
Multi-step derivations	Common but formulaic	Domain-specific
Statistical API knowledge	Basic (mean, std)	Advanced (scipy.stats)
Convention awareness	Well-established	Evolving
Asymptotic analysis	Standard	Context-dependent
Numerical edge cases	Predictable	Subtle (e.g., VaR tails)
Unit consistency	Strict	Often dimensionless

Example: VaR Calculation Failure Pure LLM generated for 95% parametric VaR:

```
VaR_95 = portfolio_value * sigma * (-1.645) # WRONG SIGN
```

Failure mode: LLM confused sign convention. In finance, VaR is reported as positive number representing potential loss, but calculation uses negative z-score. This subtle convention is not well-represented in training data (Lewkowycz et al., 2022).

Result: $R^2 = -10.05$ (predictions anticorrelated with ground truth)

Symbolic rescue: PySR discovered correct relationship including sign, achieving $R^2 = 0.9988$.

D.2 Future Research Directions

Near-Term (1-2 years):

1. GPU-Accelerated Symbolic Regression:

- Parallelize fitness evaluation across thousands of GPU cores
- GPU acceleration may offer substantial speedup over current CPU-based search
- Would potentially make hybrid approach more competitive with neural networks on speed

2. Active Learning Integration:

- Use LLM feedback to guide symbolic search
- Potentially reduce required data through uncertainty-guided sampling
- Particularly relevant for expensive experimental data

3. Multi-Scale Constant Handling:

- Logarithmic preprocessing for constants spanning ≥ 6 orders of magnitude
- Adaptive search ranges based on data scale
- May address the gravitational force failure observed in current experiments (1/15 failure rate)

Medium-Term (3-5 years):

1. PDE Discovery:

- Combine Fourier Neural Operators with symbolic regression
- Discover spatial/temporal derivative structures
- Applications: fluid dynamics, heat transfer, wave propagation

2. Hierarchical Discovery:

- Discover sub-expressions separately, then combine
- Example: $f(x) = g(h(x)) \rightarrow$ first find h , then g
- Reduces search space exponentially

3. Transfer Learning:

- Leverage formulas from related domains
- Chemical kinetics $\rightarrow$ biochemical pathways
- Classical mechanics $\rightarrow$ robotics control

Long-Term (5-10 years):

1. Automated Theory Formation:

- Long-term aspiration: not just discovering equations, but underlying principles
- Example: Discovering conservation laws from data
- Would require integration with automated theorem proving

2. Multi-Modal Scientific Discovery:

- Incorporate experimental images, diagrams, protocols
- Vision-language models for lab automation
- End-to-end hypothesis $\rightarrow$ experiment $\rightarrow$ analysis loops

3. Uncertainty-Aware Discovery:

- Discover not just $f(x)$, but $f(x) \pm \sigma(x)$
- Bayesian symbolic regression
- Applications: risk-sensitive decision making

D.3 Final Thoughts

The central lesson of this work is that **retrieval of familiar patterns may differ from structural discovery** (Kambhampati et al., 2024). On the evaluated benchmark, LLMs achieved high success rates on widely-documented equations while showing substantially lower reliability in specialized domains requiring genuine structural inference (extrapolation error statistics are in the main paper’s Five-System Comparison table).

This distinction matters for science. Discovery requires:

1. Extrapolation to novel regimes (see the main paper’s regenerated Five-System Comparison tables, Section 5, for the current per-system figures; an earlier “symbolic near-zero vs. substantially larger neural error” characterisation here did not reproduce and has been removed)
2. Functional form recovery (interpretability, insight)
3. Reliable predictions for decision-making (drug dosing, climate policy)

LLM-only approaches showed inconsistency on all three for specialized domains on the evaluated benchmark. Hybrid architectures combining LLM interfaces with symbolic engines addressed all three, at the cost of substantially longer runtime (the measured ratio is reported in the main paper’s Runtime Analysis)—an acceptable tradeoff when correctness matters.

As we integrate AI more deeply into scientific workflows, maintaining this distinction between approximation and discovery will be increasingly important. A productive direction is the orchestration of complementary capabilities: LLMs for communication and initialisation, symbolic methods for structured mathematical search, and human experts for interpretation and validation.

References

- Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Kaya Stechly, Mudit Verma, Sidhant Bhamri, Lucas Saldyt, and Anil Murthy. Llms can’t plan, but can help planning in llm-modulo frameworks. *arXiv preprint arXiv:2402.01817*, 2024.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *arXiv preprint arXiv:2206.14858*, 2022.